

## ✓ PART A: MACHINE LEARNING ENVIRONMENT SETUP

```
# 1. Verify Python version
# This prints the version of Python currently running.
import sys
print ("Python Version:", sys.version)
```

```
Python Version: 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
```

```
# 3. Import the libraries
# These imports allow us to use ML and data analysis tools.
import numpy as np # For numerical operations
import pandas as pd # For working with datasets
from sklearn import datasets # For loading built-in datasets like Iris
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

## ✓ PART B: WORKING WITH THE IRIS DATASET

```
# 1. Load the Iris dataset
# The Iris dataset is built into scikit-learn.
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris() # Load the dataset into memory

# 2. Print number of rows, number of columns, feature names

# Number of rows = number of samples
print("Number of rows:", iris.data.shape[0])

# Number of columns = number of features
print("Number of columns:", iris.data.shape[1])

# Feature names (attributes)
print("Feature Names:", iris.feature_names)

# Target names (labels/classes)
print("Target Names:", iris.target_names)

# 3. Convert the dataset into a Pandas DataFrame
```

```

# Create a DataFrame using the feature data
df = pd.DataFrame(iris.data, columns=iris.feature_names

# Add the target column (species)
df["species"] = iris.target

print("\nFirst 5 rows of the DataFrame:")
print(df.head())

# 4. Identify Dataset, Instance, Attribute, Label
# (We print examples so students can visually understand)

print("\nExample of a Dataset:")
print(df.head()) # The whole table is the dataset

print("\nExample of an Instance (one row):")
print(df.iloc[0]) # First row

print("\nExample of Attributes (columns except label):")
print(df.columns[:-1])

print("\nExample of Label (target column):")
print("species")

# 5. Why understanding the dataset is important
# (We print a simple message; explanation will be given)
print("\nUnderstanding the dataset helps us choose the c

```

Number of rows: 150

Number of columns: 4

Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'p

Target Names: ['setosa' 'versicolor' 'virginica']

First 5 rows of the DataFrame:

|   | sepal length (cm) | sepal width (cm) | ... | petal width (cm) | species |
|---|-------------------|------------------|-----|------------------|---------|
| 0 | 5.1               | 3.5              | ... | 0.2              | 0       |
| 1 | 4.9               | 3.0              | ... | 0.2              | 0       |
| 2 | 4.7               | 3.2              | ... | 0.2              | 0       |
| 3 | 4.6               | 3.1              | ... | 0.2              | 0       |
| 4 | 5.0               | 3.6              | ... | 0.2              | 0       |

[5 rows x 5 columns]

Example of a Dataset:

|   | sepal length (cm) | sepal width (cm) | ... | petal width (cm) | species |
|---|-------------------|------------------|-----|------------------|---------|
| 0 | 5.1               | 3.5              | ... | 0.2              | 0       |
| 1 | 4.9               | 3.0              | ... | 0.2              | 0       |
| 2 | 4.7               | 3.2              | ... | 0.2              | 0       |

|   |     |     |     |     |   |
|---|-----|-----|-----|-----|---|
| 3 | 4.6 | 3.1 | ... | 0.2 | 0 |
| 4 | 5.0 | 3.6 | ... | 0.2 | 0 |

[5 rows x 5 columns]

Example of an Instance (one row):

```
sepal length (cm)    5.1
sepal width (cm)     3.5
petal length (cm)    1.4
petal width (cm)     0.2
species              0.0
Name: 0, dtype: float64
```

Example of Attributes (columns except label):

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
      'petal width (cm)'],
      dtype='object')
```

Example of Label (target column):

species

Understanding the dataset helps us choose the correct ML algorithm.

## PART C: DATASET EXPLORATION

```
import pandas as pd
from sklearn.datasets import load_iris

# Load the Iris dataset again
iris = load_iris()

# Convert to DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["species"] = iris.target
```

```
# -----
# 1. Display shape, columns, data types, first/last row
#    info(), and describe()
# -----

# Shape of dataset (rows, columns)
print("Shape of dataset:", df.shape)

# Column names
print("\nColumn names:")
print(df.columns)

# Data types of each column
print("\nData types:")
print(df.dtypes)
```

```
# First few rows
print("\nFirst 5 rows:")
print(df.head())

# Last few rows
print("\nLast 5 rows:")
print(df.tail())

# Dataset information (nulls, types, memory)
print("\nDataset Info:")
print(df.info())

# Statistical summary (mean, std, min, max, etc.)
print("\nStatistical Summary:")
print(df.describe())
```

```
    'petal width (cm)', 'species'],
    dtype='object')
```

Data types:

```
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species              int64
dtype: object
```

First 5 rows:

|   | sepal length (cm) | sepal width (cm) | ... | petal width (cm) | species |
|---|-------------------|------------------|-----|------------------|---------|
| 0 | 5.1               | 3.5              | ... | 0.2              | 0       |
| 1 | 4.9               | 3.0              | ... | 0.2              | 0       |
| 2 | 4.7               | 3.2              | ... | 0.2              | 0       |
| 3 | 4.6               | 3.1              | ... | 0.2              | 0       |
| 4 | 5.0               | 3.6              | ... | 0.2              | 0       |

[5 rows x 5 columns]

Last 5 rows:

|     | sepal length (cm) | sepal width (cm) | ... | petal width (cm) | species |
|-----|-------------------|------------------|-----|------------------|---------|
| 145 | 6.7               | 3.0              | ... | 2.3              | 2       |
| 146 | 6.3               | 2.5              | ... | 1.9              | 2       |
| 147 | 6.5               | 3.0              | ... | 2.0              | 2       |
| 148 | 6.2               | 3.4              | ... | 2.3              | 2       |
| 149 | 5.9               | 3.0              | ... | 1.8              | 2       |

[5 rows x 5 columns]

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
```

```

1  sepal width (cm)    150 non-null    float64
2  petal length (cm)  150 non-null    float64
3  petal width (cm)   150 non-null    float64
4  species             150 non-null    int64

```

dtypes: float64(4), int64(1)

memory usage: 6.0 KB

None

Statistical Summary:

|       | sepal length (cm) | sepal width (cm) | ... | petal width (cm) | species    |
|-------|-------------------|------------------|-----|------------------|------------|
| count | 150.000000        | 150.000000       | ... | 150.000000       | 150.000000 |
| mean  | 5.843333          | 3.057333         | ... | 1.199333         | 1.000000   |
| std   | 0.828066          | 0.435866         | ... | 0.762238         | 0.819232   |
| min   | 4.300000          | 2.000000         | ... | 0.100000         | 0.000000   |
| 25%   | 5.100000          | 2.800000         | ... | 0.300000         | 0.000000   |
| 50%   | 5.800000          | 3.000000         | ... | 1.300000         | 1.000000   |
| 75%   | 6.400000          | 3.300000         | ... | 1.800000         | 2.000000   |
| max   | 7.900000          | 4.400000         | ... | 2.500000         | 2.000000   |

[8 rows x 5 columns]

```

# -----
# 2. Questions based on describe()
# -----

# Highest average value (mean)
highest_mean_feature = df.describe().loc["mean"].idxmax()
print("\nFeature with highest average value:", highest_mean_feature)

# Largest variation (standard deviation)
largest_std_feature = df.describe().loc["std"].idxmax()
print("Feature with largest variation (std):", largest_std_feature)

# Smallest minimum value
smallest_min_feature = df.describe().loc["min"].idxmin()
print("Feature with smallest minimum value:", smallest_min_feature)

```

Feature with highest average value: sepal length (cm)  
 Feature with largest variation (std): petal length (cm)  
 Feature with smallest minimum value: species

```

# -----
# 3. Display value counts for the target column
# -----

print("\nValue counts for species (target column):")
print(df["species"].value_counts())

```

Value counts for species (target column):

```
species
0    50
1    50
2    50
Name: count, dtype: int64
```

Double-click (or enter) to edit

```
# -----
# 4. Questions based on value counts
# -----

# Check if dataset is balanced
value_counts = df["species"].value_counts()
print("\nIs the dataset balanced?")
print(value_counts)

print("\nChecking class distribution is important before
```

```
Is the dataset balanced?
species
0    50
1    50
2    50
Name: count, dtype: int64
```

```
Checking class distribution is important before building ML models.
```

## 1. Types of Data

| Example                                       | Data Type            | Simple Justification                                           |
|-----------------------------------------------|----------------------|----------------------------------------------------------------|
| A customer database stored in MySQL           | Structured Data      | Stored in tables (rows + columns). Easy to search and analyse. |
| JSON data returned from a REST API            | Semi-structured Data | Has a structure (key-value pairs) but not fixed like tables.   |
| CCTV surveillance video                       | Unstructured Data    | No fixed format. Raw pixels, continuous frames.                |
| Emails containing headers and message bodies  | Semi-structured Data | Headers have structure, message body is free text.             |
| An Excel spreadsheet containing student marks | Structured Data      | Organized into rows and columns.                               |

b. Which type of data is easiest for traditional ML?

**Structured data** Because it is already organized into rows and columns, making it easy for algorithms to process.

c. Why is unstructured data more challenging?

Because it has **no fixed format**. Algorithms cannot directly understand images, videos, or raw text. They require **extra preprocessing**, feature extraction, or deep learning.

## 2. Identify the learning paradigm

| Scenario                                                                      | Learning Type                             | Reason                                        |
|-------------------------------------------------------------------------------|-------------------------------------------|-----------------------------------------------|
| Predicting whether a bank customer will default on a loan using labelled data | Supervised Learning                       | Labels (default / not default) are available. |
| Grouping customers by purchasing behavior without labels                      | Unsupervised Learning                     | No labels → clustering.                       |
| A robot learns to navigate a maze through rewards and penalties               | Reinforcement Learning                    | Learns by trial-and-error using rewards.      |
| Predicting house prices using historical housing data                         | Supervised Learning                       | Past prices act as labels.                    |
| Detecting unusual credit card transactions without labelled fraud cases       | Unsupervised Learning (Anomaly Detection) | No fraud labels available.                    |

### 3. Model Performance (Underfitting, Overfitting, Good Fit)

a. Identify each model

| Model | Training Accuracy | Testing Accuracy | Conclusion  | Reason                                           |
|-------|-------------------|------------------|-------------|--------------------------------------------------|
| A     | 99%               | 61%              | Overfitting | Performs well on training but poorly on testing. |
| b. B  | c. 90%            | d. 88%           | e. Good Fit | f. Training and testing accuracies are close.    |

| Model | Training Accuracy | Testing Accuracy | Conclusion             | Reason                                            |
|-------|-------------------|------------------|------------------------|---------------------------------------------------|
| g. C  | h. 58%            | i. 55%           | j. <b>Underfitting</b> | k. Both accuracies are low → model is too simple. |

b. Main disadvantage of an overfitted model

**It performs poorly on new, unseen data.** It memorizes the training data instead of learning patterns.

c. One technique to reduce overfitting

**Use regularization (L1/L2) OR Use dropout (in neural networks) OR Collect more training data OR Use cross-validation**

#### 4. Dataset Splitting

##### a. Primary purpose of dataset splits

- **Training set** → used to teach the model
- **Validation set** → used to tune hyperparameters
- **Test set** → used to evaluate final performance on unseen data

##### b. Advantages of using a separate test dataset

- Gives a **realistic estimate** of how the model performs on new data
- Prevents **over-optimistic results**
- Ensures the model is **not evaluated on data it has already seen**